# La caja negra · Reemplazo compatible

Frontier Labs · Reto 2 · Systems Design (30%) · cierre 20:00 · diseño, criterios de aceptación y arquitectura de persistencia

> **Objetivo:** sustituir un servicio HTTP heredado sin cambiar sus clientes. La unidad de corrección es **respuesta + estado posterior**: `δ(S, evento) = (S′, respuesta)`. Se preservan anomalías observadas; no se rediseñan las reglas de negocio.

## Entendimiento del problema

**Requisitos:** bearer del equipo en todas las rutas; `POST /msg`, `POST /__reset`, `GET /health` y errores de rutas/métodos. Coincidencia exacta de status y cuerpo: JSON con tipos y valores iguales, claves sin orden y arrays ordenados; no-JSON por texto exacto. No se comparan headers.

**Restricción de evaluación:** 100 tests (40 simples, 45 de 2–5 pasos y 15 de 6–12); reset antes de cada test; máximo **5 s por request**. Producción no consulta al original ni a otro reemplazo.

### Supuestos, límites y exclusiones

- **Un proceso y una instancia:** el set habla a un proceso; no hay estado compartido entre réplicas. Token del equipo configurado por entorno, no multi-tenancy.
- **Durabilidad implementada:** PostgreSQL (Neon) con una fila JSONB para el estado completo. La continuidad entre reinicios está garantizada; `reset` reescribe el estado inicial.
- **Fuera de alcance:** comandos nuevos, correcciones de anomalías heredadas, invariantes bancarias añadidas, auditoría histórica general y escalado horizontal: no ayudan a la compatibilidad exigida.

### Casos difíciles que condicionan el diseño

**E506/E507 sí mutan:** incrementan fallos del origen. El tercer fallo devuelve 422; la siguiente transferencia elegible, 409 E510. Monto, igualdad y existencia preceden al bloqueo y no incrementan fallos. Depósito desbloquea; éxito reinicia fallos; ANULA no los limpia.

**ANULA revierte una sola vez:** repetir devuelve `{"ok":false,"motivo":"ya anulada"}`. Reset borra cuentas/operaciones y reinicia IDs a 1. Conservar `"recieved"` y el espacio final de E301; no imponer saldo no-negativo.

> **Fix post-evaluación (2026-10-10):** en `server.js` se corrige `readBody` para que `413 E101` devuelva el JSON en lugar de cerrar la conexión (afectaba tests de body grande).

## Decisiones y trade-offs

Cada elección identifica alternativa, motivo y coste aceptado.

| Decisión | Alternativa descartada / por qué | Coste aceptado |
|---|---|---|
| Una instancia, monolito | Microservicios: coordinación distribuida sin beneficio para el contrato. | Sin escalado horizontal ni alta disponibilidad. |
| Dominio separado de HTTP | Reglas en rutas: acopla cambios de transporte y dificulta probar secuencias. | Dos módulos; mantener su frontera. |
| Memoria en local/tests | DB para desarrollo: infraestructura extra sin necesidad en validación local. | Reinicio pierde estado local; sin infraestructura en tests. |
| PostgreSQL JSONB de una fila (Vercel) | SQLite con volumen: driver/runtime por validar; no cubre serverless nativo. PostgreSQL multi-tabla inicial: complejidad sin réplicas requeridas. | Una fila serializa escrituras; probar locks y reinicio. |
| Compatibilidad exacta | Normalizar errores o añadir invariantes: cambia respuestas y efectos observables. | Conservar anomalías; control no criptográfico. |

**KISS/DRY:** una implementación por regla; parser y guardas internos. Sin ORM, framework FSM, DI ni event sourcing: no hay variaciones actuales que justifiquen esas capas. Revisar PostgreSQL multi-tabla o réplicas sólo ante necesidad real de escritura distribuida.

## Arquitectura y responsabilidades

**Monolito hexagonal mínimo:** cliente ↔ adaptador HTTP ↔ núcleo de estados ↔ persistencia. El adaptador autentica, enruta, limita bytes, interpreta JSON y emite status/cuerpo. El núcleo concentra parsing, dígito de control MOD 97, guardas y transiciones; no depende de HTTP.

**Estado:** cuentas (saldo/fallos), operaciones (efecto/anulada) y contadores AC/OP. Bloqueo derivado de `fallos≥3`, sin booleano duplicado. Health no muta; reset sustituye el estado completo.

> **Adaptadores de persistencia** ( `storage.js` ):
> • **Memoria** (local/tests): Map en proceso, sin infraestructura externa.
> • **PostgreSQL / Neon** (Vercel): una fila `jsonb` con `SELECT ... FOR UPDATE` dentro de una transacción. Serializa escrituras, evita estado parcial; `reset` atómico.

Ver diagrama de arquitectura para flujo, estados de cuenta/operación y topología desplegada.

## Cambios de API: detectar y adaptar

**Plan:** vigilancia externa cada 15 min hasta 20:00, token de exploración exclusivo, reset verificado y batería fija de 31 puntos. Comparar status/cuerpo y efectos posteriores; confirmar diferencias, excluir rate-limit y versionar capturas UTC. Respetar cuota.

**Adaptación controlada:** transporte → adaptador; reglas → núcleo; datos → migración de `storage.js`. Añadir regresión del cambio; actualizar SPEC/fixtures versionados; probar candidato aislado con 0 divergencias y cada paso <5 s; aprobar y desplegar. Objetivo de detección: ≤15 min + batería. No autoaprendizaje ni llamadas al original desde producción. Tras 20:00: health propio y evaluación oficial. Plan y pendientes (bitácora).

## Plan para medir y aceptar

Definir resultados esperados antes de modificar reglas. Reproducir secuencias con reset explícito y comprobar estado mediante consultas posteriores; no agrupar logs con resets ambiguos.

| Caso / prueba prevista | Resultado esperado / umbral |
|---|---|
| T1 · Entradas y precedencia | Auth ausente/incorrecta, rutas, JSON, tipos, control y límites 8191/8192 B: status/cuerpo exactos. Errores anteriores al bloqueo no suman fallos. |
| T2 · Secuencias independientes | 20 ejemplos como una secuencia; casos reservados de 2–5 y 6–12 pasos: **100% de pasos** coincidentes, sin efectos extra. |
| T3 · Fallos y desbloqueo | 3 E506/E507 → 422, 422, 422; siguiente elegible → 409. Depósito desbloquea; éxito reinicia. Confirmar saldos/fallos por comportamiento posterior. |
| T4 · ANULA y RESET | Depósito/transferencia: una reversión, segunda sin efecto. Reset elimina cuentas y OP; próximos IDs AC/OP empiezan en 1. |
| T5 · Tiempo y despliegue | En URL pública: **cada request <5 s**, cero timeouts. De 20:00 a fin de demo day: health autenticado cada 60 s, cero fallos observados. |
| T6 · Persistencia | Reinicio conserva estado; fallo interno no deja escritura parcial; E506/E507 persisten; reset atómico; locks no exceden 5 s. |

**Registro:** caso, reset, esperado/obtenido, status, cuerpo y duración por paso. **Puerta de aceptación:** 0 divergencias, 0 timeouts y 0 health fallidos observados; media/p95 no compensan un paso ≥5 s.

---

Contrato: `frontier-reto-2/README.md` . Comportamiento y evidencia: `reto2-investigacion/SPEC.md` + `experimentos.jsonl` (366 observaciones). Implementación: `reemplazo/` . Plan de aceptación separado de resultados y de propuestas aún no implementadas.